Les Modèles Algorithmiques Fondamentaux sur Structures à Accès Direct

Thème principal : Mécanismes d'accès aux collections de données et patrons de conception algorithmiques (Parcours complets systématique et conditionné, Recherche de première occurrence).

Synthèse exécutive : Ce cours pose le socle méthodologique fondamental de la conception algorithmique en BUT 1 Informatique (R1.01), indispensable pour la réussite des partiels et des SAE (Situation d'Apprentissage et d'Évaluation). Avant toute écriture de code ou de pseudo-code, l'étudiant doit analyser le contrat d'interface et le mécanisme d'accès de sa collection de données (Direct, Séquentiel ou Dynamique). Le cours détaille trois modèles canoniques réutilisables sur des structures à accès direct (tableaux, chaînes de caractères) : (1) Le parcours complet avec traitement systématique, (2) Le parcours complet avec traitement conditionné, et (3) La recherche de première occurrence. Une rigueur absolue est exigée sur la gestion des bornes d'indices (de 0 à N-1) pour éviter les débordements de mémoire ('out of bounds'), sur l'adoption du principe du pessimisme algorithmique, ainsi que sur l'obligation d'intégrer au minimum deux conditions d'arrêt pour tout algorithme de recherche.

1. Typologie et Analyse Préalable des Mécanismes d'Accès aux Données

L'étape zéro de toute démarche d'analyse algorithmique consiste à s'interroger sur la nature des accès autorisés par la structure de données sous-jacente. Si l'on manipule une structure à accès direct (un tableau statique de N éléments ou une chaîne de caractères), l'adresse mémoire de n'importe quel élément situé à l'indice 'i' est calculable immédiatement. En revanche, sur un flux séquentiel (tel qu'un fichier CSV lu ligne par ligne), l'ordinateur ne propose que trois opérations fondamentales : (1) se positionner au début du flux, (2) lire l'élément suivant, et (3) tester si la fin du flux est atteinte. Il est physiquement et logiquement impossible de sauter directement à la 50ème ligne sans avoir lu les 49 premières. Appliquer une notation par indice de type T[i] sur un fichier séquentiel constitue donc un contre-sens algorithmique qui invalide entièrement la solution proposée.

- Structure de données : Collection organisée regroupant des éléments de même type (ex: tableaux, chaînes de caractères).
- Accès Direct : Capacité matérielle ou logique d'atteindre instantanément n'importe quel élément via son indice 'i' ou une clé (ex: T[i]) en temps constant O(1), sans lire les éléments précédents.
- Accès Séquentiel : Contrainte imposant de démarrer obligatoirement en début de structure et de progresser pas à pas (début, suivant, fin) sans possibilité de saut direct (ex: fichiers texte bruts, flux CSV, listes chaînées simples).
- Accès Dynamique : Structure hybride proposant simultanément des primitives d'accès séquentiel et des mécanismes d'indexation directe.
- Incompatibilité structurelle : Interdiction absolue d'appliquer un modèle nécessitant l'accès direct sur une collection purement séquentielle sous peine de faute conceptuelle majeure.

Cas concret enseignant : *Piège récurrent en évaluation souligné par l'enseignant : rédiger un algorithme qui utilise un accès par indice 'T[i]' pour traiter un fichier texte ou un fichier CSV brut. Comme le système d'exploitation ne fournit pour ces fichiers qu'un curseur séquentiel avance-ligne, l'algorithme est sémantiquement faux et conduit directement à la note zéro.*

2. Premier Modèle : Le Parcours Complet avec Traitement Systématique

Ce modèle s'applique lorsque chaque élément d'une zone définie d'une collection doit subir une opération uniforme sans aucune exception ni filtrage (exemples : initialiser toutes les cases d'un tableau à zéro, afficher le contenu complet d'un tableau, ou convertir l'ensemble des caractères d'une chaîne en majuscules). Schéma canonique en pseudo-code : Pour i allant de BorneDébut à BorneFin Faire Traiter(Collection[i]) Fin Pour Dans un tableau de N cases en convention standard (indexation à zéro), la première case se situe à l'indice 0 et la dernière à l'indice N - 1. Le traitement est exécuté de manière indifférenciée et systématique à chaque tour de boucle.

- Exhaustivité : 100% des éléments situés entre la borne initiale et la borne finale subissent le traitement.

• Itération bornée : Utilisation obligatoire d'une boucle 'Pour' (For) car le nombre d'itérations est calculable avant le démarrage : (Fin - Début + 1).

• Gestion stricte des bornes : Pour un tableau de N éléments indicé à partir de 0, la variable d'indice varie strictement de 0 à N - 1.

• Erreur critique Out of Bounds : Tenter d'accéder à l'indice N provoque une erreur de dépassement de capacité mémoire.

Cas concret enseignant : *Exemple d'illustration vu en amphithéâtre : 'Afficher l'intégralité d'un tableau de N cases'. L'enseignant rappelle que l'erreur la plus fréquente chez les étudiants débutants consiste à écrire 'Pour i allant de 0 à N', ce qui provoque un crash immédiat ('out of bounds') lors de la tentative de lecture de la case mémoire T[N] qui n'existe pas.*

3. Second Modèle : Le Parcours Complet avec Traitement Conditionné

Le parcours complet avec traitement conditionné conserve l'inspection exhaustive du modèle précédent, mais encadre l'action de traitement par une condition de filtrage. Il est utilisé pour compter des occurrences particulières (ex: compter le nombre de voyelles dans un texte), afficher uniquement un sous-ensemble de données (ex: imprimer les nombres pairs d'un tableau), ou effectuer des calculs cumulatifs ciblés. Schéma canonique en pseudo-code : Pour i allant de BorneDébut à BorneFin Faire Si Propriété(Collection[i]) Alors Traiter(Collection[i]) Fin Si Fin Pour La condition peut être simple (ex: T[i] > 0), complexe (ex: T[i] > 0 ET T[i] mod 2 = 0), ou porter sur une comparaison de valeurs contiguës (ex: T[i] > T[i-1]). Dans ce dernier cas, la boucle doit démarrer à i = 1 (au lieu de 0) pour empêcher l'évaluation de T[-1] qui est interdite.

• Parcours exhaustif : Tous les éléments de la zone délimitée sont inspectés de la borne initiale à la borne finale.

• Traitement sélectif : L'action principale n'est déclenchée que si l'élément inspecté valide un prédicat logique (test booléen).

• Architecture algorithmique : Imbrication d'une structure alternative 'Si ... Alors' au sein d'une boucle bornée 'Pour'.

• Ajustement des bornes : Si la condition compare l'élément courant à ses voisins (i-1 ou i+1), les bornes de la boucle doivent être décalées pour éviter tout déréférencement hors limites.

Cas concret enseignant : *L'analogie de la chaîne de montage en usine donnée par le professeur : Sur une chaîne de fabrication, chaque pièce défile obligatoirement devant l'inspecteur qualité (parcours complet d'inspection de toutes les pièces), mais l'inspecteur n'appose le coup de tampon de rebut (action de traitement) que si et seulement si la pièce présente un défaut spécifique (traitement conditionné).*

4. Troisième Modèle : La Recherche de Première Occurrence & Ses Variantes

À l'inverse des parcours complets, la recherche de première occurrence s'interrompt dès que le premier élément satisfaisant le critère est localisé. L'esprit de conception repose sur le principe du pessimisme : on initialise un booléen 'trouve' à 'Faux' avant d'engager l'exploration. Tout algorithme de recherche doit obligatoirement intégrer DEUX conditions d'arrêt : 1. Le succès de la recherche (trouve = Vrai). 2. L'échec par épuisement de la collection (borne de fin atteinte). Comparatif stratégique des deux variantes d'implémentation : - Variante 1 (Sortie anticipée) : Dès que l'élément est trouvé, 'trouve' passe à 'Vrai' et l'algorithme sort immédiatement de la boucle. L'avantage majeur est que l'indice 'i' pointe exactement sur la position de l'élément trouvé. C'est la variante idéale si l'on doit retourner la position indicée exacte. - Variante 2 (Boucle Tant que globale) : La boucle s'écrit 'Tant que (i < N et NON trouve) Faire'. Si l'élément est trouvé, 'trouve' devient 'Vrai'. Cependant, la boucle effectue la mise à jour de l'indice (i <- i + 1) avant de réévaluer le test de tête. À la sortie de la boucle, si l'élément a été trouvé, l'indice 'i' a dépassé la cible de 1 et nécessite un réajustement (i - 1). Cette variante est plus simple à coder si seule l'existence booléenne nous intéresse.

• Pessimisme algorithmique : Postulat de départ supposant que la cible est absente de la collection (trouve <- Faux).

• Itération non bornée : Emploi obligatoire d'une boucle 'Tant que' (While) car le nombre d'itérations est imprévisible à l'avance.

• Double condition d'arrêt obligatoire : Un algorithme de recherche valide s'arrête SI l'élément est trouvé (succès) OU SI la collection est entièrement parcourue (échec).

• Variante 1 (Rupture immédiate) : Interruption nette dès le succès ; laisse l'indice 'i' positionné exactement sur l'élément trouvé.

• Variante 2 (Tant que global) : Condition unique en tête de boucle ('Tant que i < N et NON trouve') ; très propre syntaxiquement mais effectue une incrémentation ultime de 'i' avant de sortir.

Cas concret enseignant : *La règle d'or du professeur : 'En informatique, soyez pessimistes ! Partez toujours du principe que ce que vous cherchez n'existe pas dans la structure, et ne basculez votre indicateur à Vrai qu'au moment précis où vous le touchez.' Il martèle qu'un algorithme de recherche présenté en examen avec une seule condition d'arrêt (ex: oublier de tester la fin de tableau) recevra systématiquement la note zéro.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Est-ce qu'on peut appliquer le modèle du parcours complet avec traitement systématique sur une structure à accès dynamique ?
Réponse du professeur : Oui tout à fait. Une structure à accès dynamique propose à la fois des primitives d'accès séquentiel et des mécanismes d'accès direct. On peut donc parfaitement appliquer le modèle du parcours complet systématique en exploitant les fonctionnalités d'accès direct qu'elle met à notre disposition.

Question étudiant : Pourquoi s'embêter à nous enseigner deux variantes différentes pour la recherche de première occurrence ?
Réponse du professeur : Parce qu'elles répondent à des besoins algorithmiques distincts. La Variante 1 (avec interruption immédiate) est particulièrement avantageuse lorsqu'on a besoin de conserver l'indice exact 'i' de l'élément trouvé sans calcul supplémentaire. La Variante 2 (test global en tête de boucle) offre une structure 'Tant que' très standard et épurée au niveau du code, parfaite lorsque seule l'information d'existence (le booléen 'trouvé') est nécessaire.

Question étudiant : Comment fait-on si la condition dans un parcours complet conditionné dépend de l'élément précédent (par exemple T[i] > T[i-1]) ?
Réponse du professeur : Le modèle algorithmique fondamental demeure un parcours complet conditionné, mais la phase d'adaptation exige d'ajuster rigoureusement la borne de départ. Comme on compare T[i] à T[i-1], il faut faire démarrer la boucle à l'indice i = 1 (au lieu de 0) afin d'éviter d'évaluer T[-1] qui déclencherait un crash mémoire 'out of bounds'.

Question étudiant : Si l'on veut récupérer TOUS les indices correspondant à une valeur donnée (par exemple trouver tous les '5' d'un tableau), quel modèle doit-on utiliser ?
Réponse du professeur : Attention, il ne faut surtout pas utiliser une recherche de première occurrence ! Puisqu'il faut collecter l'ensemble des occurrences présentes dans la structure, on doit appliquer le Modèle 2 : le parcours complet avec traitement conditionné. On parcourt le tableau de 0 à N-1 et, dès que l'élément est égal à 5, on stocke ou affiche son indice.

■ Exercice & Démonstration d'Application

Énoncé : Écrire en pseudo-code algorithmique rigoureux l'algorithme permettant d'afficher tous les nombres strictement positifs d'un tableau d'entiers 'T' contenant 'N' éléments. Expliciter la démarche, le modèle retenu et la gestion des bornes.

Résolution pas à pas :
Étape 1 : Analyse du problème et sélection du modèle. Il faut inspecter l'intégralité des cases d'un tableau (structure à accès direct) de la première à la dernière. L'action d'affichage ne doit s'exécuter que si la valeur respecte le critère 'strictement supérieur à 0'. Le modèle algorithmique approprié est donc le 'Parcours complet avec traitement conditionné'. Étape 2 : Détermination des bornes d'indices. En convention informatique standard (indexation à zéro), pour un tableau de N éléments, la première case est à l'indice 0 et la dernière à l'indice N - 1. Borne initiale = 0, Borne finale = N - 1. Étape 3 : Choix de la structure de contrôle. Le nombre d'itérations est connu avant d'entrer dans la boucle -> Utilisation d'une boucle bornée 'Pour i allant de 0 à N - 1'. Étape 4 : Écriture du filtre conditionnel et du traitement. À chaque itération i, évaluer 'Si T[i] > 0 Alors' puis déclencher l'action 'Afficher(T[i])'.

Résultat : Algorithme AffichagePositifs Données T : Tableau d'entiers N : Entier (taille du tableau) Variables i : Entier Début Pour i allant de 0 à N - 1 Faire Si T[i] > 0 Alors Afficher(T[i]) Fin Si Fin Pour Fin Commentaire didactique : Cet algorithme garantit un parcours exhaustif sans risque de dépassement de capacité (l'itération s'arrête exactement à N-1) et applique le filtre sélectif demandé de manière optimale.

■ Auto-Évaluation Flash d'Examen

Q : Quel modèle algorithmique doit-on impérativement utiliser pour vérifier si un tableau d'entiers contient au moins une fois la valeur 10 ?

R : La recherche de première occurrence (ou recherche d'existence). Dès que la valeur 10 est rencontrée, le booléen 'trouvé' passe à Vrai et l'exploration peut s'interrompre.

Q : Pourquoi l'indice de fin d'un parcours complet sur un tableau de N éléments est-il égal à N - 1 et non à N ?

R : Parce qu'en informatique la numérotation des indices commence à 0. Les N cases d'un tableau s'étendent donc de l'indice 0 à l'indice N - 1. Tenter d'accéder à la case d'indice N provoque une erreur de dépassement de mémoire (out of bounds).

Q : Combien de conditions d'arrêt minimales doit comporter la structure itérative d'une recherche de première occurrence et quelles sont-elles ?

R : Elle doit comporter au minimum DEUX conditions d'arrêt : (1) Le succès de la recherche (l'élément a été trouvé) et (2) L'échec par épuisement de la collection (tous les éléments ont été visités en vain). Rédiger une recherche avec une seule condition d'arrêt constitue une erreur éliminatoire en partiel.